

Monitoring LLM Agent Behavior with Symbolic Dynamics: A System Description and Experience Report

Cognitive Workshop

June 2026

Abstract

This is a system description and experience report. We are not LLM researchers; we are engineers who built an agent harness and observed that existing monitoring tools only check *what the model says*, not *what the agent does*. We experimented with applying symbolic dynamics—an old idea from dynamical systems theory—to monitor agent tool-call sequences in real time. The approach maps tool calls and hook verdicts to a discrete symbol stream, computes Shannon entropy and a transition-graph complexity metric over rolling windows, and checks for forbidden symbol sequences under a subshift-of-finite-type constraint. It runs in approximately 200ms, requires no model internals (works with API-only LLMs), and costs nothing per check. We deployed it in a single-agent harness over 30 days (104 sessions) and observed three cases where it caught behavioral anomalies that semantic verification missed. We do *not* claim this outperforms existing methods; we have no quantitative benchmarks, no ablation studies, and no comparison against baselines. We are sharing the architecture, the math (with its known approximations), and the code in case others find the approach useful or want to improve on it.

1 Introduction

Who we are. We are not LLM researchers. We built an agent harness—a set of lifecycle hooks, state files, and safety gates that wrap around a Claude Code session—to help with long-running engineering tasks (CAD design, data analysis, document generation). After roughly one month of daily use across 104 sessions, we noticed that our semantic verification gate (a separate LLM auditing outputs) caught factual errors but systematically missed a different class of problem: the agent getting stuck in behavioral loops, or its tool-use patterns drifting into territory we had never seen before.

The gap we observed. Existing monitoring methods focus on *content*—detecting hallucinations in generated text. Semantic entropy [1], hidden-state trajectory analysis [2], and adaptive sampling [6] all answer “is this output correct?” They do not answer “is the agent behaving normally?” An agent can produce perfectly correct individual outputs while trapped in a loop, or while its action distribution slowly shifts into an unsafe regime. Content-based detectors are blind to these patterns by design.

What we tried. We experimented with **symbolic dynamics** [9]: an old mathematical framework for analyzing discrete symbol sequences. The idea is simple—treat the agent’s tool calls as a stream of symbols, and monitor the stream’s statistical properties in real time. When the Shannon entropy collapses, the agent is probably stuck. When the transition graph acquires too many new edges, the agent is probably exploring unsafely. When a known-bad symbol sequence appears, something is definitely wrong.

What this paper is (and is not). This is a system description and experience report. We describe the architecture we built, the math we used (with its known approximations noted explicitly), and three qualitative cases from our deployment. We do *not* claim superior performance over any baseline. We have no quantitative benchmarks, no ablation studies, and no comparison against existing methods. We open-sourced the code and are sharing what we learned, in case others find the approach useful.

Our design goals for the monitoring layer were:

1. **Behavioral, not semantic.** Monitor tool-call patterns and hook verdicts—externally observable events.
2. **Black-box compatible.** No model internals required. Works with any API-only LLM.
3. **Real-time and zero-cost.** Approximately 200ms per check, pure Python computation, no extra API calls.
4. **Simple enough to debug.** If the monitor itself is a black box, we have just moved the trust problem elsewhere.

2 Related Work

Semantic Entropy for Hallucination Detection. Farquhar et al. [1] introduced semantic entropy—clustering LLM outputs by bidirectional entailment and computing entropy over meaning classes—as an unsupervised hallucination detector (AUROC ~ 0.79 across 30 model–task combinations). Subsequent work improved sample efficiency via Bayesian estimation [4] and alphabetic hybrid estimation [5]. These methods require 5–20 model samples per detection and operate on output text, not agent behavior.

Geometric and Spectral Approaches. LSD [2] analyzes hidden-state trajectories across transformer layers, using cosine similarity and L2 velocity to detect hallucinations (AUROC 0.96 on TruthfulQA, single forward pass). Graph signal processing [3] computes spectral entropy over attention graphs (88.75% accuracy). Both require full access to model internals—inapplicable to API-only deployments.

Agent Behavior Analysis. The closest prior work to ours is Deng [7], which encodes ReAct agent traces into a 4-letter alphabet (Explore, Execute, Plan, Verify) and applies n-gram mining with Markov transition analysis. This demonstrates that behavioral patterns contain predictive signal for task success (trigram P-X-P lowers success rate by 10.4%), but operates post-hoc with a small fixed alphabet and does not incorporate topological invariants. ProbGuard [8] learns Discrete-Time Markov Chains from execution traces (predicting violations up to 38.66s in advance), providing PAC guarantees on learned models, but focuses on predicting known unsafe states rather than detecting novel anomalies.

Our Positioning. We do not claim to advance the state of the art. The closest prior work (Deng 2026, encoding ReAct traces as 4-letter symbols with n-gram mining) independently converged on the same core insight—that agent behavior has a statistical signature worth monitoring—and did it more rigorously. Our contribution is narrower: we combined three signals (frequency entropy, transition complexity, forbidden words) into a real-time, black-box-compatible pipeline, and we ran it in production for a month. The SFT formalism was useful as a design tool (“what should the legal behavior space look like?”), but we acknowledge the gap between the infinite-sequence theory and our 100-observation sliding window implementation.

3 Method

3.1 Event-to-Symbol Mapping

Let the agent’s observable behavior be a sequence of discrete events $e_1, e_2, e_3, \dots$ where each e_t is a tuple (tool_name, hook_verdict, domain). We define a finite alphabet $\mathcal{A} = \{s_1, s_2, \dots, s_n\}$ and a deterministic mapping $\phi : \mathcal{E} \rightarrow \mathcal{A}$ that assigns each event to a symbol.

The mapping is domain-specific: different monitoring domains define different alphabets. For example, the `hook` domain uses an alphabet of 10 symbols encoding PreToolUse check outcomes (allow, ask, deny patterns), while the `dialogue` domain uses a separate alphabet for conversation patterns. This yields K parallel observation streams $\omega^{(1)}, \omega^{(2)}, \dots, \omega^{(K)}$, one per domain.

3.2 Shannon Entropy (Symbol Frequency)

Over a rolling window of N observations in domain k , we compute the empirical symbol probabilities $p^{(k)}(s_i) = \text{count}(s_i)/N$. The Shannon entropy is:

$$H^{(k)} = - \sum_{i=1}^{n_k} p^{(k)}(s_i) \log_2 p^{(k)}(s_i) \quad (1)$$

Interpretation. $H^{(k)} \approx 0$ indicates the agent is repeating a single symbol—stuck in a loop. $H^{(k)} \approx \log_2(n_k)$ indicates maximum entropy—the agent is exploring broadly or behaving erratically. Sudden jumps in $H^{(k)}$ signal phase transitions where the behavioral regime changes abruptly. A healthy system shows moderate entropy fluctuating within a calibrated baseline.

3.3 Topological Entropy (Transition Structure)

Shannon entropy treats the symbol sequence as independent draws, discarding transition information. Real anomalies often manifest in *transitions*—the agent begins generating symbol sequences never observed during normal operation.

We construct an adjacency matrix $T^{(k)} \in \{0, 1\}^{n_k \times n_k}$ over the rolling window, where $T_{ij}^{(k)} = 1$ if symbol s_j has ever followed symbol s_i in the observation window. This defines a directed transition graph over $\mathcal{A}^{(k)}$.

The **spectral radius** $\rho(T^{(k)})$ —the largest eigenvalue of $T^{(k)}$ —bounds the growth rate of distinct symbol paths:

$$h_{\text{top}}^{(k)} \approx \log_2 \rho(T^{(k)}) \quad (2)$$

Important caveat. The formula $h_{\text{top}} \approx \log_2 \rho(T)$ is an approximation. Strictly, $h_{\text{top}} = \log \lambda_{\text{max}}$ holds for irreducible non-negative integer transition matrices of Markov SFTs; our T is a 0-1 adjacency matrix over a sliding window. The spectral radius of T is better understood as a **transition complexity index**—an upper bound on topological entropy rather than its exact value. We use it as a practical complexity metric, not as a rigorous dynamical invariant. Its behavior (monotonic growth under exploration, stability under routine operation) is empirically useful in our deployment, but we have no formal proof of its relationship to true topological entropy in this setting.

3.4 Forbidden Words (Subshift of Finite Type)

A **forbidden word** is a finite symbol sequence that must never appear in the observation stream. For example, three consecutive **DENY** verdicts in the **hook** domain signals an attack pattern or configuration error.

The set of all infinite symbol sequences over $\mathcal{A}$ that avoid a finite set $\mathcal{F}$ of forbidden words defines a **subshift of finite type** (SFT):

$$X_{\mathcal{F}} = \{\omega \in \mathcal{A}^{\mathbb{N}} : \text{no } w \in \mathcal{F} \text{ appears as a subword of } \omega\} \quad (3)$$

$X_{\mathcal{F}}$ is the **legal behavior space** for that domain. Any observed sequence falling outside $X_{\mathcal{F}}$ triggers a forbidden-word alert. Forbidden words are classified into two severity levels: P0 (block—active attack, configuration error, system compromise) and P1 (warn—statistically unusual sequence, warrants attention).

A zero-observation penalty applies to safety-critical domains: if the **hook** domain receives zero observations in a monitoring window, the domain is treated as effectively unmonitored and marked **critical**.

3.5 Stability Score

The three signals are combined into a single stability score:

$$S^{(k)} = w_1 \cdot f_1(H^{(k)}, H_{\text{baseline}}) + w_2 \cdot f_2(\rho(T^{(k)}), \rho_{\text{baseline}}) + w_3 \cdot f_3(|\mathcal{F}_{\text{hit}}|) \quad (4)$$

where f_1, f_2 are deviation functions measuring distance from calibrated baselines ($H_{\text{baseline}}, \rho_{\text{baseline}}$) and f_3 is a step function keyed to the maximum severity of any forbidden-word hit in the window. The score maps to four states:

$S^{(k)}$	Range	State	Action
≥ 0.9		ok	Normal operation
≥ 0.7		warn	Elevated, monitor
≥ 0.5		critical	Active intervention needed
< 0.5		diverging	Emergency response

All metrics use rolling windows (default $N = 100$ observations). As the window advances, old observations age out, so temporary anomalies decay naturally without manual reset.

4 Architecture

4.1 Deployment Pipeline

The system is deployed as a multi-domain pipeline integrated into the agent harness lifecycle:

1. **Observer** ($\sim 10\text{ms}$): Maps each tool call and hook verdict to domain symbols, appends to per-domain JSONL logs.
2. **Engine** ($\sim 150\text{ms}$): 8 domain engines independently compute entropy, spectral radius, and forbidden-word counts over rolling windows. Pure Python standard library, no API calls.
3. **Verdict** ($\sim 20\text{ms}$): Aggregated health states written to a shared verdict file (`symbolic_verdict.json`).
4. **Gate** ($< 5\text{ms}$): PreToolUse lifecycle hook reads current verdict before every tool call. **critical** or **diverging** state triggers DENY; **warn** triggers ASK.

Total end-to-end latency: $\sim 200\text{ms}$. Zero additional API cost.

4.2 Domain Coverage

Domain	Alphabet Size	Sensitivity
hook (safety gate outcomes)	10	High
dialogue (conversation patterns)	10	Medium
cad (CAD modeling)	9	Medium
quant (computation)	8	Medium
image (image processing)	8	Medium
window (cross-session)	Variable	Low
pic (plasma simulation)	Variable	Low
retrieval (knowledge retrieval)	Variable	Low

Each domain is independently monitored. An anomaly in one domain (e.g., **cad**) does not affect the verdict for other domains (e.g., **dialogue**), preventing cascading false alarms across unrelated subsystems.

4.3 Orthogonality to Semantic Verification

The symbolic dynamics gate (CHECK 9) operates alongside a dual-AI semantic verification gate (CHECK 16) in the same PreToolUse hook. These are orthogonal signals:

- **Symbolic dynamics**: Fast ($\sim 200\text{ms}$), statistical, behavioral. Detects loops, phase transitions, and forbidden patterns. Zero API cost.

- **Dual-AI gate:** Slower ($\sim 3\text{--}5\text{s}$), semantic, content-based. Verifies factual correctness via independent model audit. Requires API call.

The two layers catch failures that the other misses: a semantic audit cannot detect a behavioral loop, and a symbolic dynamics check cannot detect a factual error in a single output.

5 Case Studies (Illustrative Only)

The architecture has been deployed in a single-agent harness over 104 sessions spanning 30 days (May–June 2026). **These are qualitative examples, not quantitative evaluation.** We selected three cases that illustrate the kinds of anomaly each metric can detect. We do not report precision, recall, F1, or AUROC because we have no labeled test set and no baseline comparison. The examples below are meant to show *what the system can see*, not *how well it performs*.

Case 1: Entropy Collapse. During a multi-hour CAD design task, the `cad` domain’s Shannon entropy dropped from a baseline of $H \approx 1.8$ to $H \approx 0.3$ over 40 tool calls. Investigation revealed the agent repeatedly calling the same export function with minor parameter variations—a behavioral loop where each command was individually valid. The stability score transitioned from `ok` to `warn`. We manually intervened and redirected the agent.

Case 2: Transition Graph Expansion. During a retrieval task, the transition graph’s spectral radius $\rho(T)$ rose from a baseline of approximately 1.2 to approximately 3.8 over 60 observations, acquiring 14 edges absent from the calibration baseline. The agent had begun issuing novel search-fetch combinations. Shannon entropy remained within its normal range ($H \approx 1.6 \pm 0.2$), and no forbidden words were triggered—we would have missed this without the topological metric. We cannot confirm whether this was genuinely dangerous divergence or harmless exploration; the system flagged it, and we investigated.

Case 3: Forbidden-Word Intercept. Three consecutive `DENY` verdicts (a P0 forbidden word in the `hook` domain) occurred during a session where repeated tool failures triggered the compute gate. The monitor blocked further write operations before the agent compounded the error. Post-hoc review confirmed all individual tool outputs were valid—the escalating failure was only visible in the behavioral pattern.

Caveats. These cases were cherry-picked post-hoc to match the three metrics. We do not know the false-positive rate in normal operation. We have not measured whether a simple rule (“alert if the same tool is called N times consecutively”) would catch Case 1 just as well. This section exists to show the system produced non-trivial signals, not to prove it produces *good* signals.

6 Discussion and Limitations

Limitations. (1) Detection quality depends on alphabet granularity: too coarse an alphabet loses discriminative power; too fine produces sparse observations with unreliable statistics. Domain-specific tuning of alphabet size is currently manual. (2) Baseline calibration ($H_{\text{baseline}}, \rho_{\text{baseline}}$) requires a representative sample of normal behavior for each domain; domain shift over time may require recalibration. (3) Forbidden words are manually defined based on operator experience; automatic discovery from historical anomaly logs is a direction for future work. (4) The current evaluation is based on a single deployment (one agent harness, one operator); cross-system validation on public agent benchmark trajectories (e.g., SWE-bench [10]) is needed to establish generality.

Relationship to Content-Based Methods. Symbolic dynamics monitoring is not a replacement for semantic verification—it is an orthogonal layer that detects a class of anomalies (behavioral loops, phase transitions, forbidden action patterns) that content-based methods are structurally blind to. The combination of fast statistical behavioral monitoring with slower semantic content verification provides defense-in-depth for production agent deployments.

Future Work. Beyond automatic forbidden-word mining, several directions are promising: (1) cross-system validation on public agent trace datasets (SWE-bench, AgentBench) to establish quantitative benchmarks; (2) theoretical analysis of the relationship between alphabet granularity n_k and detection power as a function of observation window size N ; (3) integration with DTMC-based approaches [8] for joint statistical guarantees combining transition structure with probabilistic safety bounds; (4) extending the SFT framework to handle stochastic forbidden words with graded severity based on observed frequency rather than binary detection.

Acknowledgments

We thank the open-source LLM agent community for discussions that motivated this work. The described system, including the full 8-domain symbolic dynamics pipeline, 7-hook lifecycle architecture, and a runnable demonstration, is available as open-source software at <https://github.com/zyc1419456542/cls-cognitive-loop>.

References

- [1] S. Farquhar, J. Kossen, L. Kuhn, and Y. Gal. Detecting hallucinations in large language models using semantic entropy. *Nature*, 630:625–630, 2024.
- [2] A. Mir. The geometry of truth: Layer-wise semantic dynamics for hallucination detection. arXiv:2510.04933, 2025.
- [3] P. Noël et al. A graph signal processing framework for hallucination detection in LLMs. arXiv:2510.19117, 2025.
- [4] K. Ciosek et al. Hallucination detection on a budget: Efficient Bayesian estimation of semantic entropy. arXiv:2504.03579, 2025.
- [5] L. Pan et al. Mind the unseen mass: Unmasking LLM hallucinations via soft-hybrid alphabet estimation. arXiv:2604.19162, 2026.
- [6] xjdr-alt. Entropix: Entropy based sampling and parallel CoT decoding. <https://github.com/xjdr-alt/entropix>, 2025.
- [7] S. Deng. Your agent has a genome: Sequence-level behavioral analysis and runtime governance of LLM-powered autonomous agents. arXiv:2606.15579, 2026.
- [8] Y. Chen et al. ProbGuard: Proactive safety monitoring for LLM agents via probabilistic model checking. arXiv:2508.00500, 2025.
- [9] D. Lind and B. Marcus. *An Introduction to Symbolic Dynamics and Coding*, 2nd ed. Cambridge University Press, 2021.
- [10] J. Yang et al. SWE-agent: Agent-computer interfaces enable automated software engineering. arXiv:2405.15793, 2024.
- [11] D. Boiko et al. Emergent autonomous scientific research capabilities of large language models. arXiv:2304.05332, 2023.
- [12] X. Deng et al. Mind2Web: Towards a generalist agent for the web. arXiv:2306.06070, 2023.